Hands-on Exercise No. 4 DigiSkills 3.0 Batch-04 Full Stack Development with MERN	Total Marks: 10 Due Date: 15/10/2026

Instructions:

Please read the following instructions carefully before submitting this Hands-on Exercise:

- Use MS Word to prepare exercise solution.
- You may consult tutorials and videos if the concept is not clear.
- Your submitted exercise will not be considered/counted if:
    - It is submitted after due date.
    - It is not in the required format (doc or .docx)
    - It does not open, or file is corrupt.
    - It is copied (partial or full) from any source (websites, forums, students, etc.)

Learning Outcome:

After completing this exercise, you shall be able to:

- Create a Node.js and Express.js backend application using a proper folder structure.
- Use environment variables for application configuration.
- Connect an Express application with MongoDB Atlas using Mongoose.
- Create Mongoose schemas and models with validation and relationships.
- Implement complete CRUD operations using controllers and routes.
- Apply request validation, file upload, centralized error handling, and security middleware.
- Create user registration and login APIs using password hashing and JWT authentication.
- Test backend APIs using Postman or Thunder Client and prepare screenshots for submission.

Problem Statement

In this hands-on exercise, you are required to build a Product Inventory CRUD API using Node.js, Express.js, MongoDB, and Mongoose. You will create a full backend application with environment variables, MongoDB connection, models, controllers, routes, input validation, file upload, authentication, centralized error handling, and API testing. The objective is to strengthen your backend development skills and understand how a real CRUD-based application is structured.

You can use VS Code, Node.js, Express.js, MongoDB Atlas, Mongoose, Postman, and Thunder Client to complete this exercise.

After completing each task, take screenshots of your code, terminal output, MongoDB collection, and API testing output, and paste them into a single MS Word (.doc/.docx) file as your final solution submission.

Required API Endpoints:

```
POST   /api/auth/register
POST   /api/auth/login
GET    /api/products
GET    /api/products/:id
POST   /api/products
PUT    /api/products/:id
DELETE /api/products/:id
GET    /api/products/stats/summary
```

Tasks:

Question 1: Backend Setup, Database Connection, and Authentication (Marks 4)

Create the base backend application for a Product Inventory CRUD API.

- Part 1: Create a Node.js project and install required packages such as express, mongoose, dotenv, cors, helmet, express-rate-limit, bcryptjs, jsonwebtoken, multer, express-validator, and http-errors. Create a proper folder structure. (Marks 1)
- Part 2: Create server.js, configure Express middleware, connect MongoDB Atlas using Mongoose, and use environment variables from .env file. (Marks 1)
- Part 3: Create a User model and implement user registration with password hashing using bcryptjs. (Marks 0.75)
- Part 4: Implement user login with JWT token generation and create authentication middleware to protect private routes. (Marks 0.75)
- Part 5: Add centralized error handling, not-found middleware, and basic security middleware such as helmet, cors, and rate limiting. (Marks 0.5)

Question 2: Product Inventory CRUD API (Marks 6)

Create complete CRUD operations for products using Express, MongoDB, and Mongoose.

- Part 1: Create a Product model with fields such as name, description, price, category, stock, image, and createdBy. Add validation and useful indexes. (Marks 1)
- Part 2: Create product routes and controllers for adding, viewing all, viewing single, updating, and deleting products. Create, update, and delete routes must be protected using JWT authentication. (Marks 2)
- Part 3: Add image upload for product image using multer. Store uploaded image path in MongoDB. (Marks 1)

- Part 4: Add filtering by category, search by product name, sorting by price/date, and pagination in the get all products API. (Marks 1)
- Part 5: Add one aggregation API to show product count and average price by category. Test all APIs using Postman or Thunder Client and attach screenshots. (Marks 1)

Marking Scheme:

Question / Part	Requirement	Marks
Question 1 - Part 1	Node.js project setup, packages, and folder structure	1
Question 1 - Part 2	Express server, environment variables, and MongoDB connection	1
Question 1 - Part 3	User model and registration with password hashing	0.75
Question 1 - Part 4	Login API, JWT token, and auth middleware	0.75
Question 1 - Part 5	Centralized error handling and security middleware	0.5
Question 2 - Part 1	Product model with validation and indexes	1
Question 2 - Part 2	Complete protected CRUD routes and controllers	2
Question 2 - Part 3	Product image upload using multer	1
Question 2 - Part 4	Filtering, search, sorting, and pagination	1
Question 2 - Part 5	Aggregation API and API testing screenshots	1

NOTE: Save the code files at your end and only provide screenshots of each step in MS Word File

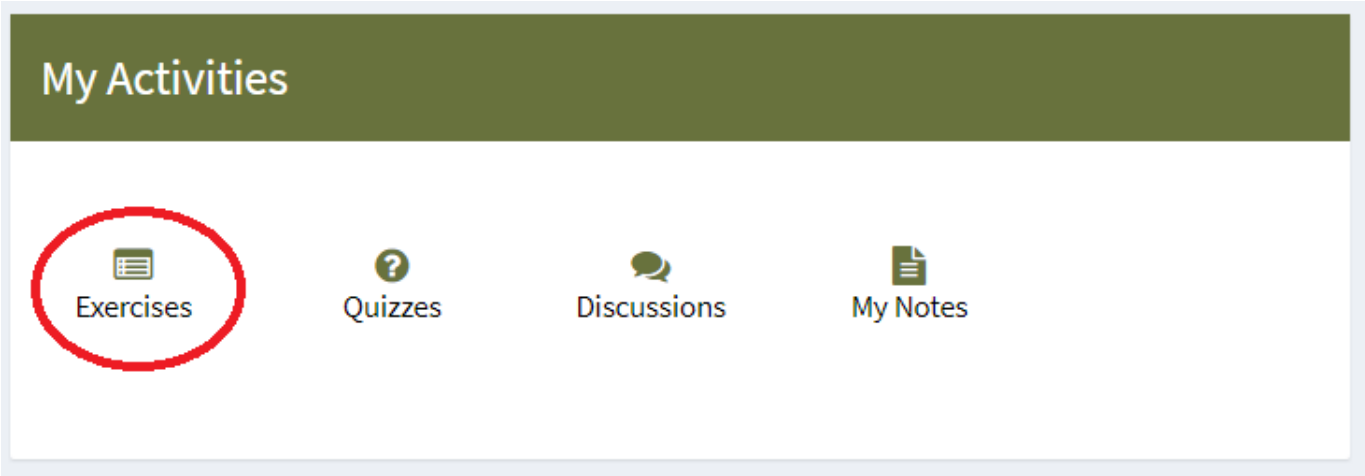
BEST OF LUCK ☺

How to submit solution file on LMS?

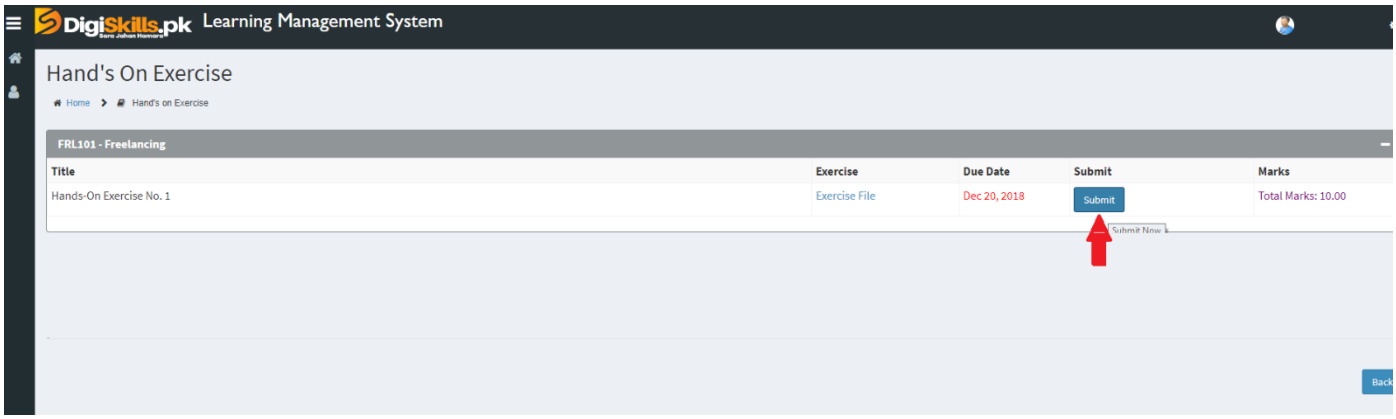
Please take the following steps for submitting your solution using LMS:

1) Login to the LMS

2) Click on the **Exercises** button within the **My Activities** section



3) Click on the submit button to upload your Solution.



4) Keep in mind to upload your Solution in .doc or .docx format